

Methodenüberladung (Method Overloading)

Aspekt	Beschreibung	Java-spezifische Konzepte/Techniken
Definition	Methodenüberladung bedeutet, dass mehrere Methoden mit dem gleichen Namen, aber unterschiedlichen Parametern existieren.	Die Methoden unterscheiden sich in Anzahl, Typ oder Reihenfolge der Parameter.
Zweck	Erhöht die Lesbarkeit und Wiederverwendbarkeit von Code, indem ähnliche Operationen unter demselben Namen gruppiert werden.	Ermöglicht die Anpassung einer Methode für verschiedene Eingabedaten ohne neuen Methodennamen.
Unterscheidung durch Parameter	Methoden müssen sich durch ihre Parameterliste unterscheiden.	Erlaubt: <code>void print(int x)</code> vs. <code>void print(double x)</code> . Nicht erlaubt: <code>void print(int x)</code> vs. <code>void print(int y)</code> (nur Name der Parameter ändert sich).
Unterschiedliche Anzahl an Parametern	Eine Methode kann mit unterschiedlicher Parameteranzahl überladen werden.	Beispiel: <code>void sum(int a, int b)</code> vs. <code>void sum(int a, int b, int c)</code> .
Unterschiedlicher Parametertyp	Methoden können sich durch unterschiedliche Datentypen ihrer Parameter unterscheiden.	Beispiel: <code>void display(int num)</code> vs. <code>void display(double num)</code> .
Unterschiedliche Reihenfolge der Parameter	Methoden können sich durch die Reihenfolge ihrer Parameter unterscheiden.	Beispiel: <code>void process(int a, String b)</code> vs. <code>void process(String b, int a)</code> .
Kein Unterschied durch Rückgabotyp	Methoden dürfen nicht nur durch ihren Rückgabotyp überladen werden.	Nicht erlaubt: <code>int calculate()</code> vs. <code>double calculate()</code> .
Methodenüberladung mit <code>varargs</code>	Methoden können überladen werden, indem eine Methode eine variable Anzahl von Argumenten (<code>varargs</code>) akzeptiert.	Beispiel: <code>void log(String... messages)</code> .

Aspekt	Beschreibung	Java-spezifische Konzepte/Techniken
Methodenüberladung und Autoboxing	Methodenüberladung berücksichtigt Autoboxing von primitiven Typen.	Beispiel: <code>void process(Integer x)</code> kann mit <code>process(10)</code> aufgerufen werden (Autoboxing von <code>int</code> zu <code>Integer</code>).
Methodenüberladung und Typumwandlung (Widening vs. Boxing)	Beim Methodenaufruf wählt der Compiler die passendste Methode anhand von Widening (Vergrößerung) und Boxing (Autoboxing).	Beispiel: <code>void show(double d)</code> wird bevorzugt gegenüber <code>void show(Integer i)</code> für <code>show(5)</code> (da <code>int</code> -> <code>double</code> ein Widening ist, während <code>int</code> -> <code>Integer</code> Boxing erfordert).
Methodenüberladung mit <code>static</code>-Methoden	Statische Methoden können ebenfalls überladen werden.	Beispiel: <code>static void display(int x)</code> vs. <code>static void display(String x)</code> .
Methodenüberladung mit <code>final</code>-Methoden	Eine <code>final</code> -Methode kann überladen, aber nicht überschrieben werden.	Beispiel: <code>final void print(int x)</code> vs. <code>void print(double x)</code> .
Methodenüberladung mit <code>private</code>-Methoden	<code>private</code> -Methoden können innerhalb derselben Klasse überladen werden.	Beispiel: <code>private void compute(int x)</code> vs. <code>private void compute(String x)</code> .
Kombination mit Vererbung	Eine Unterklasse kann Methodenüberladung nutzen, aber überladene Methoden gehören weiterhin zur Oberklasse.	Nicht Vererbung: <code>class A { void show(int x) {} void show(String x) {} }</code> → Dies sind Methoden der Klasse <code>A</code> .
Methodenüberladung vs. Methodenüberschreibung (overriding)	Methodenüberladung betrifft Methoden in derselben Klasse, während Methodenüberschreibung in einer Unterklasse geschieht.	Überladung: <code>class A { void process(int x) {} void process(String x) {} }</code> . Überschreibung: <code>class B extends A { void process(int x) {} }</code> .

Zusammenfassung

Methodenüberladung erlaubt es, mehrere Methoden mit demselben Namen, aber unterschiedlichen Parametern zu definieren. Sie verbessert die Code-Lesbarkeit, erleichtert polymorphes Verhalten und optimiert die Benutzerfreundlichkeit von APIs. Java entscheidet zur Kompilierungszeit, welche überladene Methode aufgerufen wird, basierend auf der besten Übereinstimmung der Parameter.